

Proiect 2 Scenă 3D: Sufragerie - Zi & Noapte

Documentație

1. Conceptul proiectului

Proiectul își propune realizarea unei scene 3D complexe care reprezintă o sufragerie mobilată, având ca obiectiv principal recrearea atmosferei încăperii în două momente diferite ale zilei: **ziua** și **noaptea**.

Scena conține elemente de mobilier (masă, fotoliu, canapea, televizor, vază etc.), pereți, podea și corpuri de iluminat, realizate fie prin desenare cu primitive geometrice, fie prin importarea de modele 3D din fișiere .obj.

Schimbarea momentului zilei influențează:

- tipul de iluminare utilizat,
- modul de generare al umbrelor,
- atmosfera generală a scenei.

Astfel, utilizatorul poate observa diferențe clare între iluminarea naturală (soarele – lumină direcțională) și iluminarea artificială (lustra – lumină punctiformă).

2. Elemente incluse și tehnici utilizate

Proiectul integrează majoritatea tehnicilor studiate la laborator, precum și elemente suplimentare pentru creșterea realismului.

2.1 Reprezentarea obiectelor 3D

Scena conține două tipuri de obiecte:

- **obiecte construite din primitive:** podea, pereți, tavan, unele piese de mobilier;
- **obiecte importate din fișiere .obj:** masă, fotoliu, vază, canapea, televizor, etc.

Încărcarea modelelor .obj se face folosind un loader dedicat (objloader.cpp / objloader.hpp), iar fiecare obiect este poziționat, rotit și scalat folosind matrici de model.

2.2 Iluminare

Au fost implementate **două tipuri de surse de lumină**, pentru a simula două momente diferite ale zilei:

a) Lumină punctiformă – iluminare interioară (noapte)

- Sursă de tip point light (lustra).
- Contribuie cu:
 - componentă difuză,
 - componentă speculară,
 - componentă ambientală.
- Folosită în special în modul “noapte”.

Calculul iluminării punctiforme se realizează în fragment shader folosind modelul Phong.

b) Lumină direcțională – soarele (zi)

- Sursă de tip directional light.
- Simulează lumina naturală care intră în cameră.
- Contribuie cu:
 - componentă difuză,
 - componentă speculară,
 - componentă ambientală.

Direcția soarelui este transmisă ca uniform din C++ către shader.

2.3 Umbre

Pentru a obține un nivel mai ridicat de realism, au fost implementate **două metode diferite de generare a umbrelor**, corespunzătoare fiecărei surse de lumină.

a) Umbre proiectate pe plan – pentru lumina punctiformă

Pentru iluminarea interioară, umbrele sunt generate folosind o matrice de proiecție pe plan (matrUmbra).

Obiectele sunt desenate a doua oară, transformate cu această matrice și colorate în negru semitransparent, simulând umbra pe podea.

Această metodă este simplă și eficientă pentru scene interioare.

b) Shadow Mapping – pentru lumina direcțională (soarele)

Pentru lumina soarelui a fost implementată tehnica **shadow mapping**, care presupune două etape:

1. **Depth pass** – randarea scenei din perspectiva luminii, într-un framebuffer off-screen (depthFBO), unde se salvează doar adâncimea într-o textură (depthTex).
2. **Main pass** – randarea scenei din perspectiva camerei, unde pentru fiecare fragment se verifică, folosind shadow map-ul, dacă acesta este iluminat sau se află în umbră.

În fragment shader se realizează comparația dintre adâncimea fragmentului curent și valoarea din shadow map. Pentru obținerea unor umbre mai moi, se aplică **PCF (Percentage Closer Filtering)**, prin eșantionarea mai multor pixeli vecini. Coordonatele fragmentului sunt transformate în spațiul luminii folosind matricea lightSpaceMatrix, iar valorile sunt aduse în intervalul [0,1] pentru adresarea texturii. Pentru evitarea artefactelor de tip shadow acne se aplică un bias dependent de unghiul dintre normală și direcția luminii

Această metodă permite obținerea unor umbre dinamice și realiste.

2.4 Texturare

Pentru creșterea realismului vizual, unele obiecte sunt texturate folosind coordonate de textură și sampler2D.

Texturarea este combinată cu iluminarea, astfel încât materialele reacționează corect la lumină.

2.5 Blending (amestecare)

Pentru obiectele transparente (de exemplu: becul, vaza), a fost activat blending-ul:

```
glEnable(GL_BLEND);
```

```
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

În fragment shader, canalul alpha este utilizat pentru a controla transparența, permițând vizualizarea obiectelor din spate și obținerea unui efect realist.

3. Originalitate

Elementul principal de originalitate al proiectului este **utilizarea alternativă a două surse de lumină diferite**, fiecare cu propriul sistem de generare a umbrelor, pentru a simula două momente distincte ale zilei:

- **Ziua:** lumină direcțională + shadow mapping
- **Noaptea:** lumină punctiformă + umbre proiectate pe plan

Această abordare schimbă complet atmosfera scenei și evidențiază diferențele dintre iluminarea naturală și cea artificială.

4. Capturi de ecran

- `RenderDepthPass()` - randarea scenei din perspectiva luminii, într-un framebuffer off-screen, unde se salvează doar adâncimea într-o textură (`depthTex`);

```
void RenderDepthPass()
{
    // soarele
    /*glm::vec3 sunDir = glm::normalize(glm::vec3(0.5f, -0.1f, -0.5f));*/ //E globala
    glm::vec3 lightPos = -sunDir * 800.0f;

    glm::mat4 lightView = glm::lookAt(lightPos, glm::vec3(0, 0, 200), glm::vec3(0, 0, 1));
    glm::mat4 lightProj = glm::ortho(-900.f, 900.f, -900.f, 900.f, 1.f, 2500.f);
    lightSpace = lightProj * lightView;

    glViewport(0, 0, SHW, SHH);
    glBindFramebuffer(GL_FRAMEBUFFER, depthFBO);
    glClear(GL_DEPTH_BUFFER_BIT);

    glUseProgram(depthProg);
    GLint uLS = glGetUniformLocation(depthProg, "lightSpaceMatrix");
    GLint uM = glGetUniformLocation(depthProg, "model");
    glUniformMatrix4fv(uLS, 1, GL_FALSE, glm::value_ptr(lightSpace));

    auto drawElems = [&](GLuint vao, int count, const glm::mat4& model) {
        glUniformMatrix4fv(uM, 1, GL_FALSE, glm::value_ptr(model));
        glBindVertexArray(vao);
        glDrawElements(GL_TRIANGLES, count, GL_UNSIGNED_BYTE, 0);
    };

    auto drawArr = [&](GLuint vao, int count, const glm::mat4& model) {
        glUniformMatrix4fv(uM, 1, GL_FALSE, glm::value_ptr(model));
        glBindVertexArray(vao);
        glDrawArrays(GL_TRIANGLES, 0, count);
    };

    // PODEA
    drawElems(VaoId2, 6, glm::mat4(1.0f));

    // CAMERA
    drawElems(VaoId, 54, glm::mat4(1.0f));

    // MASA
    if (nrVerticesObj1 > 0) {
        glm::mat4 modelTable =
            glm::translate(glm::mat4(1.0f), glm::vec3(0, 0, 27)) *
            glm::rotate(glm::mat4(1.0f), PI / 2, glm::vec3(1, 0, 0)) *
            glm::scale(glm::mat4(1.0f), glm::vec3(60));
        drawArr(VaoIdObj1, nrVerticesObj1, modelTable);
    }

    if (nrVerticesObj5 > 0) {
        glm::mat4 modelVase =
            glm::translate(glm::mat4(1.0f), glm::vec3(0.0f, 0.0f, 55.0f)) * //fix pe masa
            glm::rotate(glm::mat4(1.0f), PI / 2, glm::vec3(1.0f, 0.0f, 0.0f)) *
            glm::scale(glm::mat4(1.0f), glm::vec3(6.5f, 6.5f, 6.5f));
        drawArr(VaoIdObj5, nrVerticesObj5, modelVase);
    }

    if (nrVerticesObj4 > 0) {
        glm::mat4 modelArmchair = glm::translate(glm::mat4(1.0f), glm::vec3(380.0f, -250.0f, 0.0f)) *
            glm::rotate(glm::mat4(1.0f), PI / 2, glm::vec3(1.0f, 0.0f, 0.0f)) *
            glm::rotate(glm::mat4(1.0f), -3 * PI / 4, glm::vec3(0.0f, 1.0f, 0.0f)) *
            glm::scale(glm::mat4(1.0f), glm::vec3(18.0f, 18.0f, 18.0f));
        drawArr(VaoIdObj4, nrVerticesObj4, modelArmchair);
    }

    glBindVertexArray(0);
    glBindFramebuffer(GL_FRAMEBUFFER, 0);
    glViewport(0, 0, (int)width, (int)height);
}
```

- Shadow function - **determinarea fragmentelor umbrite**

Această funcție realizează testul de umbră pentru lumina direcțională folosind shadow mapping. Poziția fragmentului este transformată în spațiul luminii și adusă în intervalul [0,1] pentru adresarea texturii de adâncime. Se compară adâncimea fragmentului curent cu valoarea stocată în shadow map pentru a determina dacă lumina este blocată. Pentru evitarea artefactelor de tip shadow acne se aplică un bias dependent de unghiul dintre normală și direcția luminii. De asemenea, se utilizează PCF (Percentage Closer Filtering) prin eșantionarea mai multor pixeli vecini, obținându-se umbre mai moi și mai realiste.

```
float Shadow(vec4 fpLS, vec3 n, vec3 ldir)
{
    vec3 p = fpLS.xyz / fpLS.w;
    p = p*0.5 + 0.5;
    if(p.z > 1.0) return 0.0; //adancimea fragmentului curent (vazut de lumina)

    float bias = max(0.0025*(1.0 - dot(n, ldir)), 0.0005);

    float s=0.0;
    vec2 ts = 1.0 / textureSize(shadowMap,0);
    for(int x=-1;x<=1;x++)
    for(int y=-1;y<=1;y++){
        float d = texture(shadowMap, p.xy + vec2(x,y)*ts).r; //distanța până la
        s += (p.z - bias > d) ? 1.0 : 0.0;
        //fragmentul e mai departe decât ce e în shadow map
        //exista ceva între lumina și fragment -> e în umbra
    }
    return s/9.0;
}
```

- Calculul iluminării - **modelul Phong**

Iluminarea este calculată în fragment shader folosind modelul Phong, care include componentele ambientală, difuză și speculară. Componenta difuză este determinată prin produsul scalar dintre normală și direcția luminii, iar componenta speculară este calculată folosind vectorul de reflexie și direcția către observator. Parametrii de material, precum shininess și intensitatea speculară, permit controlul asupra aspectului suprafețelor, oferind un grad mai ridicat de realism vizual.

```
// 1. LUMINA PUNCTIFORMA (INTERIOR)
vec3 normala = normalize(Normal);
vec3 lightDirPoint = normalize(inLightPos - FragPos);
float diffPoint = max(dot(normala, lightDirPoint), 0.0);
vec3 diffusePoint = diffPoint * lightColor;

// SPECULAR
vec3 viewDirPoint = normalize(inViewPos - FragPos);
vec3 reflectDirPoint = normalize(reflect(-lightDirPoint, normala));
float specPoint = pow(max(dot(viewDirPoint, reflectDirPoint), 0.0), materialShininess); //trebuie
vec3 specularPoint = materialSpecularStrength * specPoint * lightColor; //strength variabil

// 2. LUMINA DIRECTIONALA (EXTERIOR) ---
// inLightDirSun trebuie sa fie directia Dinspre sursa
vec3 lightDirSun = normalize(-inLightDirSun);
float diffSun = max(dot(normala, lightDirSun), 0.0);
vec3 diffuseSun = diffSun * sunColor;

// SPECULAR !!! Nu sunt sigur ca lumina directionala are componenta specular
vec3 viewDirSun = normalize(inViewPos - FragPos);
vec3 reflectDirSun = normalize(reflect(-lightDirSun, normala));
float specSun = pow(max(dot(viewDirSun, reflectDirSun), 0.0), materialShininess);
vec3 sunSpecularColor = mix(sunColor, vec3(1.0), 0.7);
vec3 specularSun = materialSpecularStrength * specSun * sunSpecularColor;

// 3. COMPONENTA AMBIENTALA
float ambientStrength;

if(lightState == 0)
    ambientStrength = 0.9f;
else
    ambientStrength = 0.2f;
```

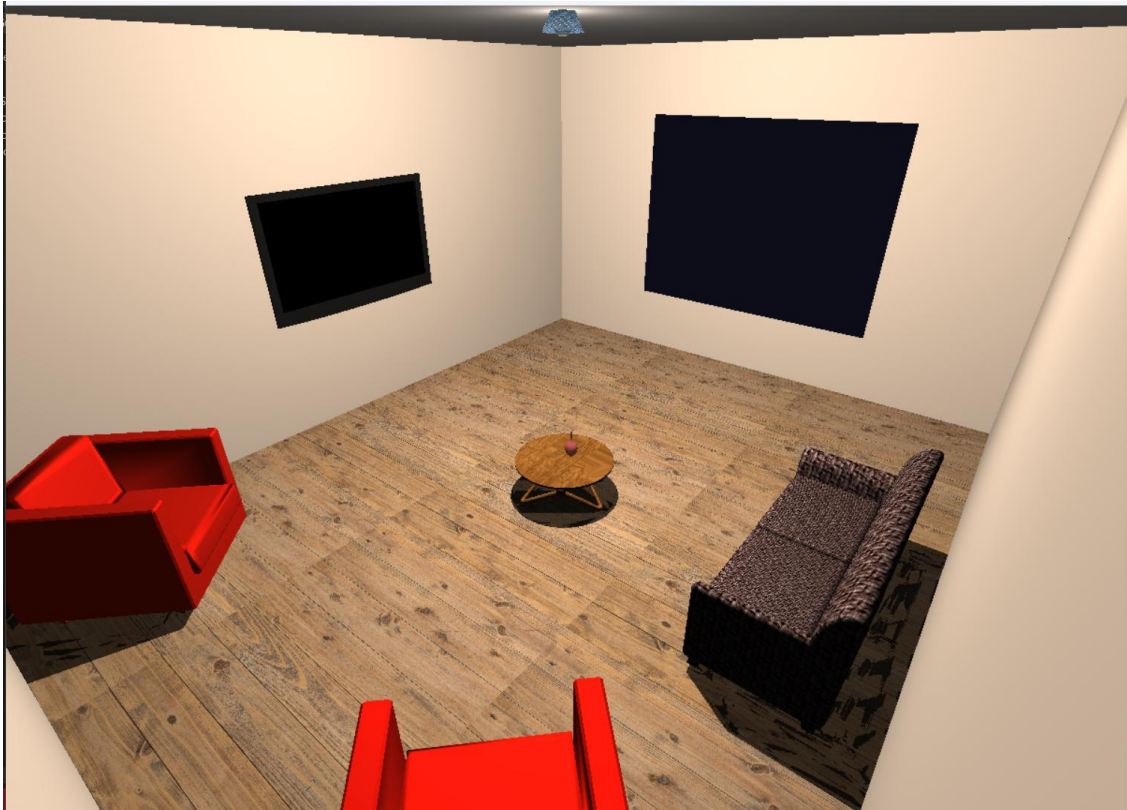
```
// Combinare Componente in Rezultat
// Adunam contributiile ambelor surse
vec3 n = normalize(Normal);
vec3 lSun = normalize(-inLightDirSun);

float shadow = (sunState==1) ? Shadow(FragPosLightSpace, n, lSun) : 0.0;
vec3 sunTerm = (diffuseSun + specularSun) * (1.0 - shadow);

vec3 result = (ambient + lightState*(diffusePoint+specularPoint) + sunState*sunTerm) * baseColor.rgb

out_Color = vec4(result, baseColor.a); //opacitate culorii, pentru obeicte transparente
```

- Rezultate



Codul sursa se află în repository-ul de la următorul link:

https://github.com/DaniLascu/Grafica_Proiect_2